Trabajo Integrador

Agente de IA con LangChain

1. Objetivo

Diseñar, implementar y justificar un **agente de IA funcional** para un caso de uso de su elección, construido con LangChain. Importa tanto el resultado como poder explicar *por qué* cada pieza está ahí y *cómo* se comporta el agente mientras resuelve una tarea.

Modalidad: grupal. Los grupos son los mismos que los de los trabajos prácticos.

Cronograma

- **Jueves 5 de noviembre:** entrega de la propuesta previa (sección 2). La cátedra la aprueba o pide ajustes; sin propuesta aprobada no se puede exponer.
- **Viernes 13 de noviembre:** clase de consultas. Cada grupo muestra el avance del proyecto y trae sus dudas.
- **Jueves 19 y viernes 20 de noviembre:** exposiciones (presentación y demo, sección 6). Se evalúa el repositorio en el estado del último commit anterior a la exposición del grupo.
- **Recuperatorio:** fecha a confirmar por la cátedra.

2. El caso de uso

El dominio es libre (finanzas personales, soporte técnico, planificación de viajes, análisis de documentos legales, asistente de estudio, etc.). Debe cumplir tres condiciones:

- El agente resuelve **tareas de varios pasos**, que no pueden resolverse con una sola llamada al modelo ni con un solo tool.
- Produce un **resultado final tangible** (un informe, un plan, una comparación fundamentada, un archivo generado), no solo una respuesta de chat.
- Está acotado: es preferible un dominio bien definido.

Propuesta previa

Antes de comenzar con el proyecto, deben preparar una propuesta que incluya:

1. **Dominio:** el negocio o problema que resuelve el agente.
2. **Usuario objetivo:** quién lo usaría y en qué situación.
3. **Ejemplo de tarea:** una tarea concreta, de varios pasos, que el agente resolverá de principio a fin.
4. **Tools mínimas:** listado de las herramientas que creen necesarias para resolver esa tarea, con una línea sobre qué hace cada una, y sus entradas y salidas.

3. Requisitos obligatorios

3.1 Agente base

- Construido con `create_agent` de LangChain.
- **No usar Deep Agents**.

3.2 Herramientas (tools)

- **Mínimo 3 tools**, todas necesarias para llegar al resultado. Si una tool se puede eliminar sin afectar en nada al caso de uso principal, no cuenta.
- **Al menos una tool propia** con implementación escrita por los integrantes.
- Al menos una tool debe interactuar con **algo externo al modelo**: web, una API, el sistema de archivos, una base de datos, generación de un documento, etc.
- Cada tool debe tener nombre, descripción y argumentos bien definidos, y manejar sus errores sin romper el flujo del agente.

3.3 Memoria de corto plazo

- Memoria conversacional mediante **checkpointer**, de modo que el agente pueda iterar sobre turnos anteriores.
- Debe poder demostrarse que funciona: la demo incluye al menos un turno que dependa de la conversación previa.

3.4 Interfaz de usuario

- Una UI **web o de escritorio** (recomiendo Streamlit) que permita conversar con el agente.
- Debe renderizar correctamente la respuesta del agente (markdown).

3.5 Visibilidad del flujo de ejecución

El usuario debe poder ver **qué hizo el agente para llegar a la respuesta** desde la UI:

- **Siempre:** cada tool invocada, con sus argumentos y un resumen del resultado, en orden.
- **Si el modelo lo expone:** el bloque de razonamiento, mostrado de forma diferenciada del contenido final. Si el proveedor elegido no lo devuelve, lo documentan en el README, lo explican en la presentación y cumplen con el punto anterior.
- Debe verse en la UI mientras el agente trabaja (*streaming*), no solo al final.

3.6 Conocimiento propio

Al menos una de estas dos opciones, a elección y con justificación:

- **(a) RAG** sobre un corpus propio (mínimo 3 documentos de su dominio): ingesta, chunking, embeddings, base vectorial y una tool de recuperación que el agente invoque cuando corresponda. La respuesta debe poder citar de qué documento viene la información. El agente debe consultar la base de conocimiento durante el flujo principal.
- **(b) Skills bajo demanda** u otro mecanismo equivalente para dar al agente conocimiento o instrucciones específicas de dominio que no estén en el modelo (por ejemplo, archivos markdown que el agente carga cuando los necesita). El agente debe hacer uso de estas skills en su flujo principal.

Los embeddings pueden hacerse sin costo con el tier gratuito de Google (recomiendo el modelo `gemini-embedding-001`).

Importante: las herramientas de conocimiento ocupan contexto del agente. Tengan en cuenta el contexto del que disponen a la hora de dimensionar los chunks que retornará el RAG / la cantidad de información aportada por las skills.

3.7 Middleware

- Si usan middleware ya hecho de LangChain, que sea **el mínimo indispensable** y con justificación explícita (qué problema resuelve y por qué no se resolvió con código propio).

4. Proveedor de LLM

Cada equipo consigue su propio proveedor. Si ya poseen API Keys que estén pagando, pueden utilizarlas. Opciones gratuitas válidas: tiers gratuitos de Google Gemini, OpenRouter, Groq o Mistral, o modelos locales con LM Studio u Ollama.

- El modelo **debe soportar tool calling**. Verifiquen esto antes de elegirlo.
- Los tiers gratuitos tienen límites de peticiones y latencias altas. Diseñen contemplando eso: manejo de errores 429 / reintentos con espera, y un caso de demo que no dependa de demasiadas llamadas.
- Idealmente, elijan un modelo de **razonamiento**, y un proveedor que les permita **recuperar los bloques de razonamiento**, para mostrarlo en la UI.
- Algunos modelos han sido entrenados sobre dominios específicos. Investiguen cada opción disponible antes de decidir definitivamente qué modelo usar.

5. Justificación de componentes

Los integrantes del equipo deben ser capaces de justificar la presencia de cada componente (cada tool, la memoria, el mecanismo de conocimiento, la UI, el proveedor y modelo elegidos, cualquier middleware). Durante la exposición, la cátedra puede preguntarle a cualquier integrante por cualquier componente, no solo por el que implementó. Esa defensa se califica en forma individual (sección 7).

6. Entregables

1. **Repositorio de GitHub** con:
   - Código ejecutable, organizado y comentado donde corresponda.
   - `README.md` con instalación, configuración (`.env.example` sin claves reales) y cómo ejecutarlo en pasos claros.
   - `requirements.txt` (o equivalente) completo.
   - Si hay corpus o ingesta de documentos, instrucciones y scripts para reproducirlos.
2. **Presentación**: descripción del caso de uso, modelo elegido (y por qué), diagrama del flujo del agente, explicación y justificación de componentes (sección 5), limitaciones conocidas y qué mejorarían.
3. **Demo**: ejecución en vivo donde se vea una tarea completa, el flujo de tools, el razonamiento si existe y un turno que use memoria. Debe mostrarse el resultado final producido por el agente.

7. Evaluación

El trabajo integrador se califica de 0 a 100 y se aprueba con 60 puntos o más, que corresponden a una nota de 6. Su nota pesa el 40 % de la nota final de la materia (0,4 × PI) y tiene una instancia de recuperación (ver cronograma).

Los criterios 1 a 6 se califican por grupo y suman hasta 80 puntos. El criterio 7 se califica por integrante y suma hasta 20; por eso la nota final del integrador puede variar entre integrantes del mismo grupo. Un requisito obligatorio de la sección 3 que no esté presente deja su criterio en «No logrado».

Logrado otorga el puntaje completo del criterio; Parcial, entre un tercio y dos tercios; No logrado, cero.

Criterio	Logrado	Parcial	No logrado
1. Caso de uso y agente (10 pts)	La tarea requiere varios pasos reales y termina en un resultado tangible. Agente construido con `create_agent`, sin Deep Agents.	La tarea se resuelve con pocos pasos o el resultado es poco más que una respuesta de chat.	La tarea se resuelve con una sola llamada, o no se usa `create_agent`.
2. Tools (20 pts)	Tres o más tools, todas necesarias; al menos una propia y una que interactúa con algo externo. Nombres, descripciones y argumentos claros; los errores no rompen el flujo.	Se cumplen los mínimos, pero alguna tool sobra, está mal descripta o un error corta el flujo.	Menos de tres tools necesarias, o falta la propia o la externa.
3. Conocimiento propio (15 pts)	RAG o skills integrados al flujo principal y justificados. En RAG, la respuesta cita el documento de origen; el tamaño de lo recuperado está pensado para el contexto.	El mecanismo funciona pero se usa poco en el flujo principal, no cita la fuente o satura el contexto.	No hay mecanismo de conocimiento propio o el agente no lo usa.
4. Memoria, UI y visibilidad del flujo (15 pts)	La memoria con checkpointer se demuestra con un turno que depende del anterior. La UI renderiza markdown y muestra en streaming cada tool con sus argumentos y resultado, y el razonamiento si el modelo lo expone.	Falta alguno de los elementos (streaming, orden de las tools, razonamiento disponible y no mostrado) o la memoria no se demuestra.	No hay memoria, o la UI no muestra qué hizo el agente.
5. Repositorio y reproducibilidad (10 pts)	Se instala y ejecuta siguiendo el README; `.env.example` sin claves; dependencias completas; la ingesta del corpus es reproducible.	Hace falta resolver pasos no documentados para ejecutarlo.	No se puede ejecutar a partir del repositorio, o hay claves reales commiteadas.
6. Presentación y demo (10 pts)	Cubre todo lo pedido en la sección 6 (incluido el diagrama del flujo y las limitaciones) y la demo muestra una tarea completa con su resultado final.	Falta alguno de los puntos de la presentación, o la demo queda incompleta.	No hay demo funcionando.
7. Defensa individual (20 pts)	Explica y justifica cualquier componente que se le pregunte, incluidos los que no implementó.	Justifica los componentes propios pero no los del resto del grupo.	No puede explicar cómo funciona el agente.